

Introduction

Hardware description languages occupy an unusual position in the study of machine-generated code. Unlike general-purpose software, an HDL artifact is not “correct” merely because it parses and passes unit tests: it must compile under strict language semantics, elaborate and synthesize to a physically realisable netlist, simulate correctly against reference stimuli, respect protocol and timing constraints, and do all of this within finite hardware resources. A generated VHDL entity that simulates correctly but infers unintended latches, or that meets functionality while tripling the LUT budget, is not a success by any engineering standard. Evaluation of LLM-assisted hardware design must therefore be *multi-dimensional*, *toolchain-backed* and *statistically disciplined* — three properties that current practice in the literature only partially delivers.

The dominant evaluation idiom today remains demonstration: a small number of hand-selected designs, generated once, inspected by eye, and reported without confidence intervals, without controlled prompting, and without a mechanism for third parties to re-run the study. Even the commendable recent wave of Verilog benchmarks (Section 2) generally reports pass/fail functional correctness as the sole signal, leaving structural quality, resource cost and failure aetiology unmeasured. The consequence is that claims across papers, tools and model versions are not commensurable, and progress over time cannot be tracked.

hdlevel is our response. It is deliberately *research infrastructure* rather than an application: a fixed experimental apparatus into which heterogeneous HDL generators can be inserted and measured under identical conditions. The design is governed by five architecture decision records (ADRs) that we summarise in Section 4; the two most consequential are that *inference is decoupled from evaluation* (a generator is injected behind a `ModelProvider` interface, so evaluation code is never modified to add a model) and that *benchmarks are pure data* (structured metadata plus verified reference HDL, versioned independently of the platform).

This paper makes the following contributions:

1. **A versioned benchmark suite** (v1) of 18 VHDL design tasks across seven categories — arithmetic, finite-state machines, communication protocols, memory, processor components, DSP and control — each specified as structured metadata with functional requirements, an expected interface, verification properties and a verified reference implementation, and each assigned an objective difficulty score (Section 5).
2. **A reproducible evaluation harness** that executes one identical eleven-stage pipeline for every (model, prompt, benchmark, trial) tuple, with graceful degradation when toolchain components are absent and strict enforcement in CI (Section 6).
3. **A metric framework** spanning functional, structural, resource and process dimensions, together with a canonical nine-class failure taxonomy assigned by first failing stage (Section 6.2).
4. **A controlled baseline study** of 216 evaluations demonstrating that the harness produces stable, statistically separable rankings, that difficulty tiers modulate outcomes, and that toolchain-backed grading materially changes conclusions relative to static analysis alone (Sections 7–9).
5. **A claim–evidence discipline** in which every capability claim is mapped to a CI-checkable artifact, including mutation campaigns that measure the detection coverage of the verification stack itself (Section 10).

We emphasise a methodological point that shapes the entire paper: the reported baseline intentionally evaluates *control* providers — a deterministic golden reference and three synthetic-fidelity generators — rather than a commercial LLM. This isolates the measurement instrument from any particular model API, makes the entire study reproducible without credentials, and establishes calibration anchors (a known-perfect upper bound and known-degraded baselines) against which real-model results can later be interpreted. Evaluating a real model requires changing one configuration line.

Related Work

Benchmarking LLM code generation. HumanEval and MBPP established the now-standard paradigm for software: large versioned task suites, automatic unit-test grading, and pass@ k metrics. Their influence is visible in nearly every subsequent code-generation study, and their central lesson — that shared, versioned, automatically gradable suites make claims commensurable — directly motivates hdlevel. Hardware, however, breaks the paradigm’s implicit assumption that execution of a test suite suffices as an oracle: HDL grading additionally requires compilation under strict semantics, synthesis to a netlist, and event-driven simulation.

LLMs for HDL generation. Early work such as DAVE framed English-to-Verilog as machine translation. Thakur et al. fine-tuned and benchmarked LLMs for Verilog generation, later extended in VeriGen. Chip-Chat explored conversational co-design of a microprocessor, and ChipNeMo adapted foundation models to industrial EDA workflows. These works demonstrate feasibility but largely evaluate on bespoke task sets with pass/fail or human judgement.

Verilog benchmark suites. VerilogEval and RTLLM are the closest relatives of our benchmark suite: both provide versioned Verilog tasks with automatic functional grading. hdlevel differs in four respects. First, it targets VHDL-2008, which is essentially unserved by existing suites despite its industrial prevalence in aerospace, defence and European design houses. Second, grading is toolchain-backed end-to-end: GHDL compilation and simulation, Yosys synthesis, and resource extraction are first-class pipeline stages rather than optional add-ons. Third, every benchmark carries a declared complexity vector from which an objective difficulty score is computed, enabling difficulty-stratified analysis (RQ3). Fourth, the platform ships the full experimental apparatus — registry, statistics, report generation — not only the tasks.

Repair and agentic flows. RTLFixer and related work show that syntax- and simulation-guided repair loops improve outcomes. hdlevel treats such interventions as *experimental conditions*: prompt strategies and repair policies are plugins measured by the same harness (RQ4), rather than baked into the evaluator.

Open-source EDA. The measurement stack builds on Yosys, GHDL, and the Yosys/nextpnr flow; formal checks use bounded model checking via SymbiYosys. The availability of a complete open toolchain is what makes a fully reproducible hardware evaluation platform possible at all.

Reproducibility. Our registry-first design — every artifact a deterministic build product of recorded experiments — follows the reproducibility programme articulated for machine learning by Pineau et al., and our interval-first reporting follows standard guidance on binomial interval estimation.

Research Questions and Hypotheses

The platform is built against four questions fixed in advance in a living research specification:

Can a single config-driven harness fairly and reproducibly compare heterogeneous HDL generators — different models, prompting strategies, repair policies — across a broad design taxonomy?

Which quantitative signals beyond functional pass/fail — compilation, synthesis, simulation, structural complexity, resource utilisation, failure class — best discriminate between generators?

How does generation quality change as a function of an *objective* benchmark difficulty score, and is that score predictive?

Do prompting strategies, self-repair, retrieval augmentation and optimization objectives

measurably improve outcomes, with what effect size and confidence?

Four falsifiable hypotheses accompany them: **H1**, that a config-driven harness with decoupled inference yields stable, reproducible rankings under re-run and re-seed; **H2**, that structure- and resource-level metrics reveal quality differences invisible to pass/fail; **H3**, that pass rate declines monotonically with objective difficulty; and **H4**, that chain-of-thought and self-repair help more on moderate/hard benchmarks than trivial ones (an interaction effect). The baseline study reported here speaks to H1–H3; H4 requires the real-model and multi-prompt studies scheduled next (Section 13).

Platform Architecture

Layered, plugin-based design

hdlevel is organised as strictly layered components, each extensible by plugin registration:

Layer	Role
configs/	typed, YAML-loaded experiment definitions
ModelProvider	inference: reference synthetic API
PromptStrategy	direct CoT few-shot RAG repair
benchmarks/v1	pure-data suite + reference HDL
toolchain adapters	GHDL / Yosys, graceful degradation
metrics	static structural + synthesis resource
EvaluationHarness	the fixed eleven-stage pipeline
ExperimentDB	SQLite registry, full provenance
leaderboard / reports	aggregation with confidence intervals
publication / website	generated artifacts, never hand-edited

Architecture decision records

Five ADRs govern the design and are versioned with the code. *ADR-0001 (config over code)*: an experiment is a declarative document — models × prompts × benchmark selector × toolchain/verification settings × trials × seed — so that studies are diffable, reviewable artifacts. *ADR-0002 (inference decoupled from evaluation)*: generators implement a narrow ModelProvider interface; the harness never contains model-specific logic, which is the structural guarantee behind RQ1. *ADR-0003 (benchmarks as pure data)*: a benchmark contains no executable platform code, so the suite can be released and cited as a standalone dataset. *ADR-0004 (graceful toolchain degradation)*: absent tools cause stages to record skipped, never silent passes; CI sets HDLEVAL_REQUIRE_TOOLS=1, making degradation an error. *ADR-0005 (SQLite registry)*: a single local, append-only, machine-readable store of every evaluation record with environment fingerprint, git commit, seeds, inference configuration, retries and artifact hashes.

Reproducibility as an architectural property

Reproducibility is not a documentation promise but a build property: python reproduce.py rebuilds the benchmark suite, executes the experiment, and regenerates every leaderboard, table, figure, report, preprint input and website page from the registry. Determinism is achieved through seeded providers, disk-cached real-model responses, environment fingerprinting, independent benchmark versioning, and content-hashed DAG caching of intermediate artifacts. The platform’s publication pipeline enforces a stronger invariant still: no number in a generated document is typed by a human.

Benchmark Suite Design

Benchmarks as structured metadata

Suite v1 comprises 18 benchmarks across seven categories (Table 1; full listing in Appendix 15). Each benchmark is a YAML document containing a natural-language specification, an explicit list of functional requirements, the expected entity interface, semantic tags, a declared complexity vector, verification properties (e.g. `overflow_detection`, `mutual_exclusion` for the FIFO), and a path to a verified reference implementation in VHDL-2008. Reference implementations serve three roles: they establish an attainable upper bound (the `reference-golden` provider replays them), they supply oracle behaviour for simulation-based grading, and they prove by construction that every task is solvable within the declared interface.

Benchmark suite v1 by category.

Category	#	Representative tasks
arithmetic	4	adder, ALU, comparator, multiplier
communication	3	UART TX, SPI master, I2C controller
memory	3	sync. FIFO, register file, RAM
fsm	2	sequence detector, traffic controller
dsp	2	4-tap FIR, moving-average filter
processor	2	ALU control, branch predictor
control	2	debouncer, PWM generator
total	18	

An objective difficulty model

Each benchmark declares a complexity vector $\mathbf{c} = (c_1, \dots, c_7)$ over seven dimensions: state complexity, arithmetic complexity, concurrency, hierarchy depth, timing constraints, interface count and control complexity. The difficulty score is the fixed weighted sum

$$D(\mathbf{c}) = \frac{100}{Z} \sum_{k=1}^7 w_k c_k, \quad Z = \sum_{k=1}^7 w_k c_k^{\max}, \quad \text{label}\{eq:difficulty\}$$

normalised to $[0, 100]$ and bucketed into five tiers (trivial/easy/moderate/hard/expert). The weights w_k are fixed per suite version and published with it; they are *a priori* engineering judgements, deliberately not fitted to any model’s performance, so that difficulty is a property of the artefact rather than of a particular generator. In suite v1 the resulting distribution is 1 trivial, 8 easy, 6 moderate and 3 hard benchmarks (no v1 task reaches the expert tier). Empirical calibration of w_k against observed outcomes is explicitly deferred and flagged as a threat to validity (Section 11).

Versioning and citability

The suite is versioned independently of the platform (`benchmarks/v1`), with the intent that any revision — added tasks, corrected references, re-weighted difficulty — produces a new citable version, and that any historical result names the exact suite it was measured against. Because benchmarks are pure data (ADR-0003), the suite can be consumed by other harnesses without importing `hdlevel` code.

Evaluation Methodology

The identical procedure

Every (model, prompt, benchmark, trial) tuple executes exactly one pipeline:

```
$$\begin{split} \text{load spec} \rightarrow \text{build prompt} \rightarrow \text{infer} \\ \rightarrow \text{parse} \rightarrow \text{compile} \rightarrow \text{synthesize} \\ \rightarrow \text{simulate} \rightarrow \text{metrics} \rightarrow \text{properties} \end{split}
```

$\rightarrow \text{classify} \rightarrow \text{record} \rightarrow \text{end}$

No stage is ever specialised per model. Compilation and simulation use GHDL against oracle-derived expected vectors; synthesis uses Yosys, whose `stat` output supplies resource metrics. When a tool is absent the corresponding stages record skipped and functional correctness falls back to a documented, clearly-labelled static check; under `HDLEVEL_REQUIRE_TOOLS=1` (as in CI and in all results reported here) degradation is instead a hard error, so every number in this paper is toolchain-backed.

Metric framework

Metrics are computed at four levels, chosen so that RQ2 can be answered rather than assumed:

- **Functional:** compilation, synthesis and simulation success; per-property verification results; overall functional correctness (a passing simulation against reference vectors).
- **Structural** (tool-free, deterministic): hierarchy depth, combinational complexity, inferred-latch risk, reset strategy, FSM state count, module decomposition, fan-out and modularity.
- **Resource:** LUTs, flip-flops, DSP and BRAM utilisation, estimated maximum clock frequency, and area efficiency, extracted from Yosys.
- **Process:** inference latency, token usage, retry count and wall-clock duration.

Canonical failure taxonomy

Each failed evaluation receives exactly one class, assigned by the *first failing stage*: `no_code_generated`, `syntax_error`, `compilation_error`, `synthesis_failure`, `simulation_failure`, `protocol_violation`, `timing_failure`, `verification_failure`, `optimization_failure`. First-stage attribution makes classes mutually exclusive and gives the taxonomy a causal reading: a record classified `simulation_failure` necessarily compiled and synthesized.

Statistical methodology

The platform never reports a point estimate without its interval. For a pass count x out of n , we report the Wilson score interval

$$\frac{\hat{p} + \frac{z^2}{2n}}{1 + \frac{z^2}{n}} \pm z \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}, \quad \hat{p} = x/n, \quad z = 1.96, \quad \text{label}\{eq:wilson\}$$

preferred over the Wald interval for its coverage behaviour at small n and extreme $\hat{p}$ — both endemic in per-category slices. Seeded repeated trials (three per tuple in the baseline) provide variance estimates. For paired model comparisons across the same benchmarks the platform applies McNemar’s test on discordant pass/fail pairs, and bootstrap resampling for deltas of continuous metrics (latency, resource counts).

Experimental Setup

Controlled providers as calibration anchors

Experiment `baseline-v1` evaluates four providers, all deterministic given the recorded seed:

- **reference-golden:** replays the verified reference implementation for each benchmark. By construction this is the attainable upper bound; any harness that fails to score it at 100% is broken. It functions as a positive control.

- `synthetic-high`, `synthetic-mid`, `synthetic-low`: seeded generators that emit the reference implementation degraded with fidelity-dependent fault injection — truncated architectures, corrupted syntax, or omitted output — modelling generators of decreasing quality. They function as graded negative controls with *known-ordered* true quality.

This design measures the measurement instrument. If the harness is sound, it must (a) score the positive control perfectly, (b) recover the known quality ordering of the synthetic providers with separable confidence intervals, and (c) attribute their failures to the correct stages. All three predictions are testable and are borne out in Section 8. A real model is evaluated by switching the provider to `anthropic` in the experiment config; responses are disk-cached so that published real-model results remain reproducible without re-querying the API.

Configuration and environment

The baseline runs all four providers with the `direct` prompt strategy over the full 18-benchmark suite, three seeded trials each: $4 \times 18 \times 3 = 216$ evaluations, of which 69 fail. The toolchain is GHDL (VHDL-2008) and Yosys, hard-required. Per-run provenance captured in the registry includes OS and Python versions, git commit, seeds, full inference configuration and retry counts. Experiments run on commodity Linux CI runners; no GPU is required for the baseline.

Results

Model	Prompt	n	Pass rate	95% CI	Compile	Synth	Latency (s)	Tokens
reference-golden	direct	54	1.000	[0.934, 1.000]	1.000	1.000	0.100	320.7
synthetic-high	direct	54	0.722	[0.591, 0.824]	0.722	0.722	1.736	298.6
synthetic-mid	direct	54	0.556	[0.424, 0.680]	0.556	0.556	1.746	265.7
synthetic-low	direct	54	0.444	[0.320, 0.576]	0.444	0.444	1.718	259.7

Overall pass rate per provider with 95% Wilson intervals. The harness scores the positive control at exactly 1.0 and recovers the designed quality ordering of the synthetic controls.

The harness recovers ground truth (H1, RQ1)

Table [tab:leaderboard] and Figure 1 present the headline result. The positive control scores 54/54; the three synthetic controls score 0.722, 0.556 and 0.444, exactly in their designed fidelity order. The Wilson intervals of the strongest and weakest synthetic configurations ([0.591, 0.824] vs. [0.320, 0.576]) are disjoint, so at $n = 54$ the instrument statistically separates the extremes of the control ladder; the adjacent-pair intervals overlap, quantifying honestly how much resolution 54 evaluations buy. Re-running the experiment under the recorded seeds reproduces the registry bit-for-bit, and re-seeding preserves the ranking, supporting H1. The identical-pipeline design means this stability is structural rather than

incidental: there is no code path by which a provider could receive different grading.

Latency and token metrics illustrate the process dimension: the golden provider replays instantly (0.10s) while synthetic providers model realistic inference delays (≈ 1.7 s), and average token counts decrease monotonically with fidelity (320.7 down to 259.7) as degradation truncates output — an early example of a non-functional signal tracking quality (RQ2).

Category structure (RQ2)

Figure [fig:bycategory] decomposes pass rate by category (the full table with intervals appears in Appendix 16). Two observations stand out. First, categories are far from exchangeable: arithmetic and FSM tasks are passed at high rates by most controls, while DSP and memory tasks discriminate sharply (*synthetic-mid* and *-low* both score 0.0 on DSP). Second, degradation interacts with category in ways a single overall number conceals: *synthetic-high* outperforms *-mid* overall yet scores 0.0 on memory where *-mid* scores 0.667, because its particular fault-injection profile happens to break FIFO/RAM control logic. For real models the analogous phenomenon — category-specific blind spots invisible in aggregate — is precisely what per-category reporting with intervals is designed to expose.

Difficulty scaling (H3, RQ3)

Pass rate by objective difficulty tier. The positive control is flat at 1.0; synthetic controls show tier-dependent structure that is only partially monotone, indicating the v1 difficulty weighting needs empirical calibration.

Figure 2 plots pass rate against difficulty tier (per-tier table in Appendix 16). H3 predicts monotone decline. The evidence is mixed and worth reporting precisely *because* it is mixed: *synthetic-high* declines from trivial/easy toward moderate ($1.0 \rightarrow 0.875 \rightarrow 0.333$) but recovers on hard (1.0); *synthetic-low* shows the reverse pattern on moderate (0.0) versus hard (0.667). Two honest readings coexist. Mechanically, synthetic fault injection is seeded per benchmark, not difficulty-aware, so the controls are not obliged to respect the tier ordering — the tier analysis here validates the *machinery* (stratified aggregation, small- n intervals such as [0.439, 1.0] for the single trivial benchmark), not H3 itself. Methodologically, the wide intervals at $n = 3$ and $n = 9$ per cell demonstrate exactly why the platform refuses to report point estimates alone. A genuine test of H3 awaits real-model data, and the v1 weights w_k in Eq. [eq:difficulty] are candidates for empirical recalibration once such data exist.

Hypothesis H2: signals beyond pass/fail

In the baseline, compile, synthesis and simulation rates coincide with pass rate for every provider (Table [tab:leaderboard]) — by construction, since synthetic degradation produces code that either compiles and is correct, or fails early. For real models these rates are expected to diverge (code that compiles but simulates incorrectly), and the structural and resource metric layers exist precisely to capture quality differences among *passing* designs. The baseline already shows process-level separation (tokens, latency); a full test of H2 is future work with real generators.

Error Analysis

Failure-class distribution per provider (of 54 evaluations each). All 69 failures are genuine toolchain rejections or absent output; no failure class beyond compilation is reached, confirming first-stage attribution.

Failure classification, baseline-v1.

Model	compilation_error	no_code	None
reference-golden	0	0	54
synthetic-high	15	0	39
synthetic-mid	21	3	30
synthetic-low	21	9	24
total	57	12	147

Table 2 and Figure 3 classify all 69 failures. Two classes occur: `compilation_error` (57; GHDL rejecting the emitted RTL) and `no_code_generated` (12), with the share of absent output growing as fidelity drops ($0 \rightarrow 3 \rightarrow 9$). The distribution carries a methodological argument: because GHDL and Yosys actually ran, these 57 compilation failures are *ground-truth* rejections. A static-fallback-only evaluation — the degraded mode the harness permits when tools are absent — would have graded a substantial fraction of these records as plausible code, materially overstating correctness for exactly the lower-fidelity configurations where accuracy matters most. This is the platform’s central empirical case for toolchain-backed verification as a non-negotiable default (ADR-0004’s strict mode), and a caution against HDL evaluations that grade on parseability or pattern-matching alone.

Verification Soundness and the Claim–Evidence Discipline

A benchmark platform’s results are only as credible as its verification stack, so hdeval’s companion pipeline subjects that stack itself to adversarial measurement, under a repository-wide rule: *if CI does not prove it, the documentation does not say it*. Every capability claim is mapped, in a maintained claim–evidence table, to the artifact or job that backs it. Elements of this discipline include:

- **Negative testing.** A seeded stuck-carry bug — constructed to be invisible to model-level checks — must cause GHDL simulation to fail. This demonstrates the harness can detect wrongness, not merely confirm correctness.
- **Mutation campaigns.** Eight independent defect classes are injected into known-good RTL; the first detecting stage is recorded per class and *escapes are reported honestly* rather than suppressed, yielding an empirical detection-coverage map of the verification

stages.

- **Bounded formal verification.** Class invariants are checked on the emitted RTL with SymbiYosys bounded model checking (depth 24) on a synthesizable formal wrapper of the actual DUT. Results are vocabulary-controlled: a bounded pass is recorded as “BMC pass (depth N)”, never as “proven”; unbounded k -induction proof is future work and is never claimed. Model-level checks (exhaustive up to 10-bit operands, bounded beyond) are labelled `model_*` and explicitly make no claim about the VHDL.
- **Measured vs. estimated QoR.** Every quality-of-result record carries a `source` field; only numbers from actual place-and-route (nextpnr on iCE40 : SB_LUT4/SB_DFF counts, achieved $F_{\max}$, deterministic at seed 1) count as measurements, and the timing estimator’s error against these measurements is itself published as a calibration artifact (MAPE and scale ratio).
- **Weakest-link confidence.** Per-run confidence aggregates stage scores by minimum rather than average, so a single unverified stage caps overall confidence — a deliberately conservative policy.

Oracle independence is established not by re-implementing oracles but by the combination of (a) GHDL executing the *emitted RTL* — an independent artifact — against oracle-derived values, and (b) the mutation campaign demonstrating that RTL-side deviations are caught. There is exactly one golden model per design class, eliminating the consistency hazards of duplicated oracles.

Threats to Validity

Construct validity. The baseline measures the harness with synthetic controls, not any commercial model; nothing here should be read as a claim about real LLM capability. The difficulty weights w_k are fixed a priori and not yet empirically calibrated; tier analyses (Figure 2) should be interpreted accordingly. Functional correctness is defined by reference test vectors and declared properties, which cannot exhaust behaviour; formal results are bounded, not proofs.

Internal validity. Reference implementations are verified by construction and by the mutation campaign, but not exhaustively for wide operands (exhaustive checking collapses past ~10-bit operands; wider designs receive bounded checks, labelled as such; embedded testbenches cap at 2048 vectors). Where tools are absent the harness degrades to a weaker static fallback — excluded from all results here, but a hazard for third-party reruns that skip the provided Docker image.

External validity. Suite v1 is 18 single-design VHDL benchmarks of at most moderate hierarchy; generalisation to large hierarchical designs, other HDLs, and industrial constraints is untested. Per-cell sample sizes in stratified analyses are small; the platform surfaces this through intervals rather than hiding it.

Statistical conclusion validity. With $n = 54$ per configuration, only the extreme configurations separate at the 95% level; adjacent comparisons need more trials or paired tests (Section 6.4), which the registry supports.

Reproducibility Statement

Every number, table and figure in this paper regenerates from the experiment registry via `python reproduce.py`; the publication pipeline forbids hand-typed results by construction. The repository ships the versioned benchmark suite, all experiment configs, the SQLite registry for `baseline-v1`, a Dockerfile pinning the GHDL/Yosys toolchain, seeded providers, and CI that runs the toolchain for real (`HDLEVAL_REQUIRE_TOOLS=1`) with 66 passing tests at 89% line coverage. Real-model responses, when evaluated, are disk-cached and committed so published results do not depend on API availability. The platform records per-run environment fingerprints sufficient to reproduce any single evaluation given the same model and toolchain.

Future Work

Five directions are scheduled. **(1) Real-model studies:** multi-model benchmarking through the anthropic provider and successors, testing H2–H4 with paired statistics and cached responses; the immediate next experiment crosses prompting strategies (direct, chain-of-thought, few-shot, retrieval-augmented, self-repair) with difficulty tiers to test the H4 interaction. **(2) Formal methods at scale:** from depth-bounded BMC toward k -induction and equivalence checking against reference implementations, upgrading “BMC pass” to “proven” only where an unbounded argument exists. **(3) Learning-based interventions:** retrieval augmentation over the knowledge base and reinforcement-learning optimization of prompt and repair policies, evaluated as plugins under the unchanged harness. **(4) Benchmark growth and calibration:** expert-tier tasks, empirical recalibration of difficulty weights against real-model outcomes, and release of the suite as an independently citable dataset. **(5) Language generality:** Verilog, SystemVerilog and Chisel backends through the modular code-generation interface, with cross-language difficulty equating.

Conclusion

hdlevel demonstrates that the evaluation of LLM-assisted hardware design can be conducted with the same experimental discipline expected elsewhere in machine learning: versioned benchmarks, a fixed toolchain-backed harness, multi-level metrics, canonical failure attribution, interval-first statistics, and one-command reproducibility. The controlled baseline validates the instrument — perfect recovery of a positive control, correct ordering of graded negative controls, and honest exposure of its own resolution limits — and the error analysis supplies concrete evidence that static-only grading would overstate correctness precisely where it matters. The platform, suite and registry are open; we invite the community to break them, extend them, and measure against them.

99

M. Chen et al., “Evaluating Large Language Models Trained on Code,” *arXiv:2107.03374*, 2021.

J. Austin et al., “Program Synthesis with Large Language Models,” *arXiv:2108.07732*, 2021.

H. Pearce, B. Tan, and R. Karri, “DAVE: Deriving Automatically Verilog from English,” in *Proc. ACM/IEEE Workshop on Machine Learning for CAD (MLCAD)*, 2020.

S. Thakur et al., “Benchmarking Large Language Models for Automated Verilog RTL Code Generation,” in *Proc. Design, Automation and Test in Europe (DATE)*, 2023.

S. Thakur et al., “VeriGen: A Large Language Model for Verilog Code Generation,” *ACM Trans. Design Automation of Electronic Systems*, 2024.

J. Blocklove, S. Garg, R. Karri, and H. Pearce, “Chip-Chat: Challenges and Opportunities in Conversational Hardware Design,” in *Proc. ACM/IEEE Workshop on Machine Learning for CAD (MLCAD)*, 2023.

M. Liu et al., “ChipNeMo: Domain-Adapted LLMs for Chip Design,” *arXiv:2311.00176*, 2023.

M. Liu et al., “VerilogEval: Evaluating Large Language Models for Verilog Code Generation,” in *Proc. IEEE/ACM Int. Conf. on Computer-Aided Design (ICCAD)*, 2023.

Y. Lu, S. Liu, Q. Zhang, and Z. Xie, “RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Models,” in *Proc. Asia and South Pacific Design Automation Conf. (ASP-DAC)*, 2024.

Y. Tsai, M. Liu, and H. Ren, “RTLFixer: Automatically Fixing RTL Syntax Errors with Large Language Models,” in *Proc. Design Automation Conf. (DAC)*, 2024.

C. Wolf, “Yosys Open SYnthesis Suite,” 2013. <https://yosyshq.net/yosys/>

- T. Gingold, “GHDL: A VHDL 2008/93/87 Simulator,” <https://ghdl.github.io/ghdl/>
- D. Shah et al., “Yosys+nextpnr: An Open Source Framework from Verilog to Bitstream for Commercial FPGAs,” in *Proc. IEEE Symp. on Field-Programmable Custom Computing Machines (FCCM)*, 2019.
- A. Biere, A. Cimatti, E. Clarke, and Y. Zhu, “Symbolic Model Checking without BDDs,” in *Proc. Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 1999.
- J. Bachrach et al., “Chisel: Constructing Hardware in a Scala Embedded Language,” in *Proc. Design Automation Conf. (DAC)*, 2012.
- E. B. Wilson, “Probable Inference, the Law of Succession, and Statistical Inference,” *J. American Statistical Association*, 22(158):209–212, 1927.
- L. D. Brown, T. T. Cai, and A. DasGupta, “Interval Estimation for a Binomial Proportion,” *Statistical Science*, 16(2):101–133, 2001.
- Q. McNemar, “Note on the Sampling Error of the Difference between Correlated Proportions or Percentages,” *Psychometrika*, 12(2):153–157, 1947.
- B. Efron, “Bootstrap Methods: Another Look at the Jackknife,” *Annals of Statistics*, 7(1):1–26, 1979.
- J. Pineau et al., “Improving Reproducibility in Machine Learning Research,” *J. Machine Learning Research*, 22(164):1–20, 2021.

Benchmark Suite v1 Listing

All 18 benchmarks in suite v1.

ID	Category	Title
arith_adder8	arithmetic	8-bit ripple-carry adder
arith_alu8	arithmetic	8-bit ALU (5 ops)
arith_cmp8	arithmetic	8-bit magnitude comparator
arith_mul8	arithmetic	8×8 unsigned multiplier
comm_uart_tx	communication	UART transmitter (8N1)
comm_spi_master	communication	SPI master (mode 0)
comm_i2c_ctrl	communication	I2C bit-level controller
mem_fifo	memory	synchronous FIFO (depth 16)
mem_regfile	memory	32×32 register file (2R1W)
mem_sync_ram	memory	single-port RAM (256×8)
fsm_seqdet	fsm	sequence detector (1011, ovl.)
fsm_traffic	fsm	traffic-light controller
dsp_fir4	dsp	4-tap FIR filter
dsp_movavg	dsp	moving-average filter (win. 4)
proc_alu_ctrl	processor	ALU control decoder
proc_branch_pred	processor	2-bit saturating branch pred.
ctrl_debounce	control	switch debouncer
ctrl_pwm	control	PWM generator (8-bit duty)

Each benchmark declares: specification, functional requirements, expected entity interface, tags, a seven-dimensional complexity vector (Eq. [eq:difficulty]), verification properties, and a verified VHDL-2008 reference implementation.

Stratified Results with Intervals

Pass rate by category (95% Wilson CIs), baseline-v1.
 Abbreviations: ref = reference-golden, s-h/m/l =
 synthetic-high/mid/low.

Category	Model	<i>n</i>	Pass	95% CI
arithmetic	ref	12	1.000	[0.758, 1.000]
	s-h	12	1.000	[0.758, 1.000]
	s-m	12	0.750	[0.468, 0.911]
	s-l	12	0.500	[0.254, 0.746]
communication	ref	9	1.000	[0.701, 1.000]
	s-h	9	1.000	[0.701, 1.000]
	s-m	9	0.333	[0.121, 0.646]
	s-l	9	0.667	[0.354, 0.879]
control	ref	6	1.000	[0.610, 1.000]
	s-h	6	1.000	[0.610, 1.000]
	s-m	6	0.000	[0.000, 0.390]
	s-l	6	1.000	[0.610, 1.000]
dsp	ref	6	1.000	[0.610, 1.000]
	s-h	6	0.500	[0.188, 0.812]
	s-m	6	0.000	[0.000, 0.390]
	s-l	6	0.000	[0.000, 0.390]
fsm	ref	6	1.000	[0.610, 1.000]
	s-h	6	1.000	[0.610, 1.000]
	s-m	6	1.000	[0.610, 1.000]
	s-l	6	0.500	[0.188, 0.812]
memory	ref	9	1.000	[0.701, 1.000]
	s-h	9	0.000	[0.000, 0.299]
	s-m	9	0.667	[0.354, 0.879]
	s-l	9	0.000	[0.000, 0.299]
processor	ref	6	1.000	[0.610, 1.000]
	s-h	6	0.500	[0.188, 0.812]
	s-m	6	1.000	[0.610, 1.000]
	s-l	6	0.500	[0.188, 0.812]

Pass rate by difficulty tier (95% Wilson CIs),
 baseline-v1.

Tier	Model	<i>n</i>	Pass	95% CI
trivial	ref	3	1.000	[0.439, 1.000]
	s-h	3	1.000	[0.439, 1.000]
	s-m	3	1.000	[0.439, 1.000]
	s-l	3	0.000	[0.000, 0.562]
easy	ref	24	1.000	[0.862, 1.000]
	s-h	24	0.875	[0.690, 0.957]
	s-m	24	0.500	[0.314, 0.686]
	s-l	24	0.750	[0.551, 0.880]
moderate	ref	18	1.000	[0.824, 1.000]
	s-h	18	0.333	[0.163, 0.563]
	s-m	18	0.667	[0.438, 0.837]
	s-l	18	0.000	[0.000, 0.176]
hard	ref	9	1.000	[0.701, 1.000]
	s-h	9	1.000	[0.701, 1.000]
	s-m	9	0.333	[0.121, 0.646]
	s-l	9	0.667	[0.354, 0.879]

